iOS ET SDK 集成文档 (英文版)

Prerequisite

- xcode 13.0 or above version.
- iOS 12.0 or above devices.

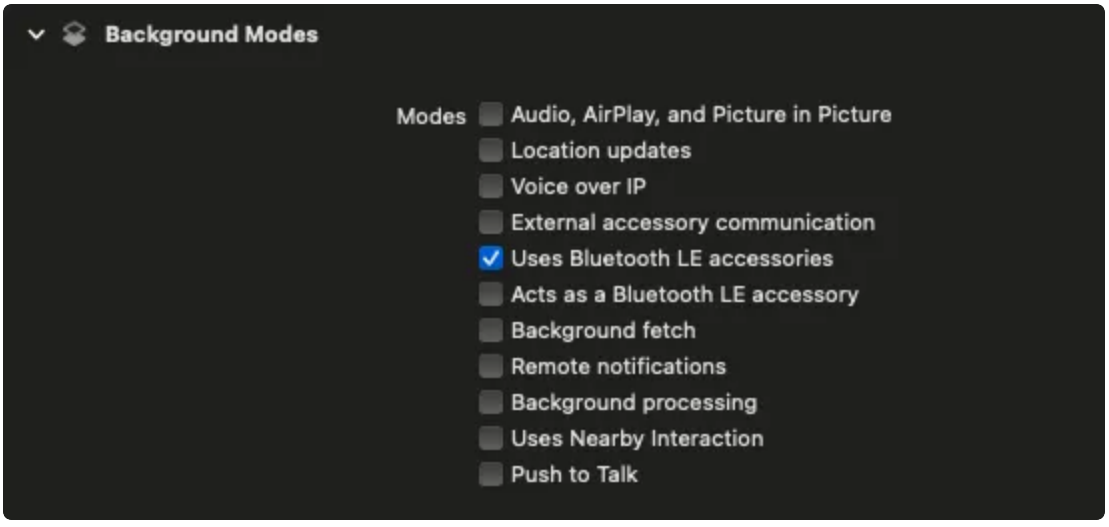
Import foundation

1. Add device permissions for the project.

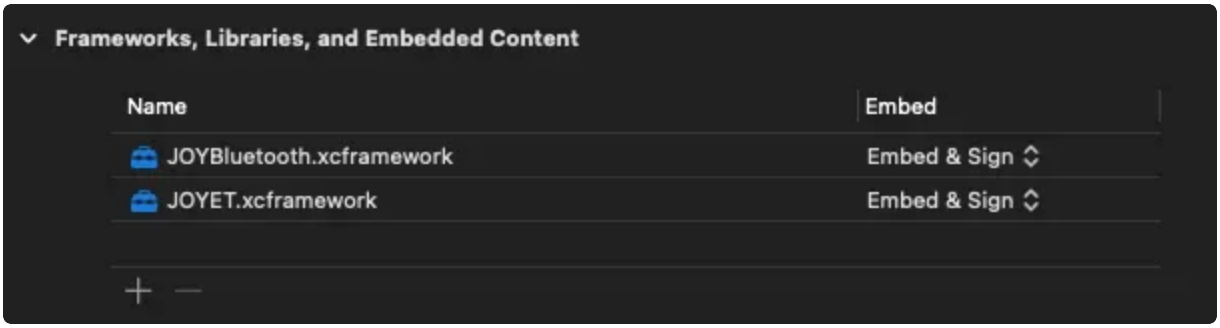
   Open the `info.plist` file in the project navigation bar, edit the attribute list, and add the following attributes:

key	type	value
Privacy – Bluetooth Always Usage Description	String	Do you allow applications to use Bluetooth to connect devices for data transmission?
Privacy – Bluetooth Peripheral Usage Description	String	Do you allow applications to use Bluetooth to connect devices for data transmission?

2. Add `Background Modes` and check the box `Users Bluetooth LE accessories`.



3. Add the ET module SDK into the project.



`JOYET.xcframework` is the ET function module SDK, and `JOYBluetooth.xcframework` is the bluetooth function module SDK;

> **note:** JOYBluetooth.xcframework is optional, of course, you can also use the system Bluetooth interface to achieve some Bluetooth functions.

Module introduction

In `JOYET.xcframework` , the functions of each class and protocol are as follows:

`JOYETCore` is ET function management class, and functional interfaces can be called through instance objects of this class;

`JOYETDelegate` is ET function related callback, which can obtain measurement data through this protocol;

`JOYETModel` is a measurement result model that can be obtained through measurement completion callback and can be used to display measurement result;

Functional realization

This section introduces all the functions of using the ET SDK in the application.

Create Bluetooth

Import header file, declare and initialize `JOYBluetoothEngine` , and comply with `JOYBluetoothDelegate` protocol:

```objective-c
// import header file
#import <JOYBluetooth/JOYBluetooth.h>

// comply with protocol
@interface xxx () <JOYBluetoothDelegate>

// declare JOYBluetoothEngine object
@property (nonatomic, strong) JOYBluetoothEngine *bluetooth;

@end

// Create bluetooth engine
self.bluetooth = [JOYBluetoothEngine engine];

// add delegate object
[self.bluetooth addDelegate:self];
```

Scan peripheral

Before using the search peripheral function, it is recommended to implement the `callback for bluetooth state update` to obtain the current bluetooth state:

```objective-c
// central device bluetooth state update
- (void)didCentralUpdateState:(CBManagerState)state {
    if (state == CBManagerStatePoweredOn) {
        // available
    } else {
        // unavailable
    }
}
```

Scan for peripherals by using `bluetooth` and get the peripherals through callback:

```objective-c
// scan for peripherals
[self.bluetooth scan];

// found peripherals, you can get the peripherals for displaying
- (void)didDiscoverPeripheral:(CBPeripheral *)peripheral AD:(NSDictionary *)AD {
    if (![peripheral.name containsString:@"DET"]) {
        return;
    }
    // Do something
}
```

Connect

Select a peripheral from the searched peripherals to connect:

```objective-c
// connect the peripheral
[self.bluetooth connect:peripheral];

// successfully connected peripheral
- (void)didConnectPeripheral:(CBPeripheral *)peripheral {

}

// failed to connect peripheral
- (void)didFailToConnectPeripheral:(CBPeripheral *)peripheral
                             error:(NSError *)error {

}
```

When the device is successfully connected, it is necessary to discover services:

```objective-c
// successfully connected peripheral
- (void)didConnectPeripheral:(CBPeripheral *)peripheral {
    [peripheral discoverServices:nil];
}
```

```objective-c
// discover services
- (void)didDiscoverServices:(CBPeripheral *)peripheral error:(NSError *)error {
    for (CBService *service in peripheral.services) {
        [peripheral discoverCharacteristics:nil forService:service];
    }
}
```

You just need to listen for the callback that discovers characteristics.

```objective-c
// discovered characteristics
- (void)didDiscoverCharacteristics:(CBPeripheral *)peripheral service:(CBService *)service error:(NSError *)error {

}
```

Disconnect

Manually disconnect the connection, and notify users by `callback for disconnecting` :

```objective-c
// manually disconnect the connection
[self.bluetooth disconnect:peripheral];

// callback for disconnecting
- (void)didDisconnectPeripheral:(CBPeripheral *)peripheral error:(NSError *)error {

}
```

> **note:** when the central device actively calls the interface to disconnect, error is nil; when disconnected due to device malfunction, error is not nil.

Reconnect

When the peripheral is disconnected abnormally, if automatic reconnection is required, call the interface as follows:

```Objective-C
// disconnected
- (void)didDisconnectPeripheral:(CBPeripheral *)peripheral error:(NSError *)error {
    // scan when errror is not nil,
    if (error != nil) {
        [self.bluetooth scan];
    }
}

// found and connect
- (void)didDiscoverPeripheral:(CBPeripheral *)peripheral AD:(NSDictionary *)AD {
    if (...) {
        [self.bluetooth connect:peripheral];
    }
}
```

Create ET

Import header file, declare and initialize `JOYETCore` , and comply with `JOYETDelegate` protocol:

```Objective-C
// import header file
#import <JOYET/JOYET.h>

// comply with protocol
@interface JOYETViewController () <JOYETDelegate>

// declare JOYETCore object
@property (nonatomic, strong) JOYETCore *core;

@end

// Create ET Core
self.core = [JOYETCore create:self];
```

Set peripheral

When receiving a callback for discovering characteristics, set the peripheral of ET core:

note: Must be set after receiving the callback for connected peripheral.

```objective-c
// connected peripheral
- (void)didConnectPeripheral:(CBPeripheral *)peripheral {
    self.core.peripheral = peripheral;
}
```

Set characteristics

```objective-c
- (void)didDiscoverCharacteristics:(CBPeripheral *)peripheral service:(CBService *)service error:(NSError *)error {
    if (peripheral != self.selectedDevice.peripheral) {
        return;
    }
    if (![service.UUID.UUIDString isEqualToString:@"FFF0"]) {
        return;
    }
    for (CBCharacteristic *c in service.characteristics) {
        if ([c.UUID.UUIDString isEqualToString:@"FFF1"]) {
            // FFF1 notify !!!
            [peripheral setNotifyValue:YES forCharacteristic:c];
        }
        else if ([c.UUID.UUIDString isEqualToString:@"FFF2"]) {
            // FFF2 write !!!
            self.core.writeCharacteristic = c;
        }
    }
}
```

Disconnected

When disconnected from the device, the `disconnected` method needs to be called:

```objective-c
// disconnected
- (void)didDisconnectPeripheral:(CBPeripheral *)peripheral error:(NSError *)error {
    [self.core disconnected];
}
```

Handle message

When receiving a message from the device, the `handle:` method needs to be called:

```Objective-C
// characteristic value update
- (void)didUpdateValue:(CBPeripheral *)peripheral characteristic:(CBCharacteristic *)characteristic error:(NSError *)error {
    [self.core handle:characteristic.value];
}

```

Measure ET

> **note:** ET measurement does not have an interface. When measuring on peripheral as required, the central device will receive measurement related callbacks, and users will make corresponding interface changes and display measurement result based on the callbacks.
>
> After the measurement is completed, return an array containing current measurement data and historical measurement data, with a maximum of 30 entries.

```Objective-C
// finish measure
- (void)didFinishMeasure:(JOYETCore *)core list:(NSArray<JOYETModel *> *)list {

}
```